Etapa 5. Separar persistencia y transacciones

1. Interfaces de Repository (Definición en el Dominio)

Las interfaces viven en la capa de Dominio y operan únicamente sobre las Raíces de Agregado (Aggregate Roots). Los métodos expresan la intención del negocio sin exponer detalles técnicos como sentencias SQL o uniones de tablas.

Repository	**Necesidad del dominio**	**Métodos propuestos**	**Objeto que recupera o guarda**
ReservaRepository	Recuperar una reserva pendiente para validar su pago o liberarla si vence el plazo de 15 minutos.	obtenerPorId(id: String): Reserva pendientesVencidas(ahora: DateTime): List<Reserva> guardar(reserva: Reserva)	Reserva (Aggregate Root)
SesionRepository	Verificar el historial de sesiones confirmadas de un tutor para prevenir traslapes y registrar la evidencia.	obtenerPorId(id: String): Sesion obtenerSesionesConfirmadasPorTutor(tutorId: String): List<Sesion> guardar(sesion: Sesion)	Sesion (Aggregate Root)
DisputaRepository	Gestionar el expediente de un desacuerdo y asegurar que no se permita	obtenerPorId(id: String): Disputa actualizar(disputa: Disputa)	Disputa (Aggregate Root)

	cerrar sin evidencia.		
PagoRepository	Gestionar el desglose del cobro (20/80), penalizaciones y registrar reembolsos sustentados.	porReserva(reservaId: String): Pago guardar(pago: Pago)	Pago (Aggregate Root)
TutorRepository	Administrar la disponibilidad por materia y registrar faltas para la regla de suspensión (BR-0018).	obtenerPorId(id: String): Tutor guardar(tutor: Tutor)	Tutor (Aggregate Root)

2. Implementación en Infraestructura

La implementación real de estas interfaces reside en la capa de Infraestructura.

- **Aislamiento:** El Dominio no sabe si los datos se guardan en SQL Server, PostgreSQL (Supabase) o memoria.
- **Mapeo:** Las clases de infraestructura (ej. SqlReservaRepository) reciben la entidad del dominio, mapean sus *Value Objects* (como Dinero o PlazoDePago) a columnas de la base de datos mediante un ORM (como Entity Framework Core o Hibernate) y construyen las consultas SQL necesarias (ej. SELECT * FROM Reservas WHERE ...).

3. Gestión de Transacciones (Unit of Work)

El patrón *Unit of Work* (UoW) asegura que las operaciones de los casos de uso se traten como una transacción atómica: o se completan todas juntas, o no se aplica ninguna.

Definición del bloque transaccional:

- **¿Dónde inicia?** La transacción inicia explícitamente en la Service Layer (Capa de Aplicación) antes de cargar los objetos uow.begin(). Esto permite adquirir bloqueos de concurrencia (como la exclusión de agenda del tutor) para evitar traslapes o dobles cobros antes de siquiera leer la base de datos.
- **¿Qué incluye?** Agrupa todas las llamadas a los métodos guardar() de los repositorios que modifiquen el estado en un solo bloque. Al terminar con éxito, se llama a uow.commit().
- **¿Qué ocurre ante un error?** Si se viola una regla de negocio (ej. falla la BR-0020 de evidencia), falla la conexión a la red, o se produce un error de concurrencia, el bloque lanza una excepción y se ejecuta uow.rollback(). Resultado: La base de datos queda intacta, evitando datos huérfanos o estados inconsistentes (como registrar un pago descontado sin haber creado la sesión).

Ejemplo de operaciones que deben completarse juntas (Flujo CU-01b - Confirmar Pago):

Cuando el estudiante confirma el pago de una tutoría, la transacción incluye coordinar 3 agregados distintos al mismo tiempo. Todo debe guardarse junto:

1. **Reserva:** Cambia su estado a Confirmada (si no ha vencido).
2. **Sesion:** Nace una nueva sesión en estado Programada.
3. **Pago:** Se guarda el importe validado y se calcula el DesglosePago (20% comisión).

(Si el servidor se cae al guardar la sesión, el pago y la reserva se revierten gracias al Unit of Work).